CAMSW100-000
API Manual

Version: V1.0

Note: This document is suitable for CAMSW100-000 firmware revision V1.3.1 or later

Revision History

Version	Prepare	Verify	Date	Notes
V1.0	Hornet		2025.09.19	Original version

Contents

1 Introduction

1.1 Preparation

This section takes a third party control device windows 10 as an example. You may also use other control devices.

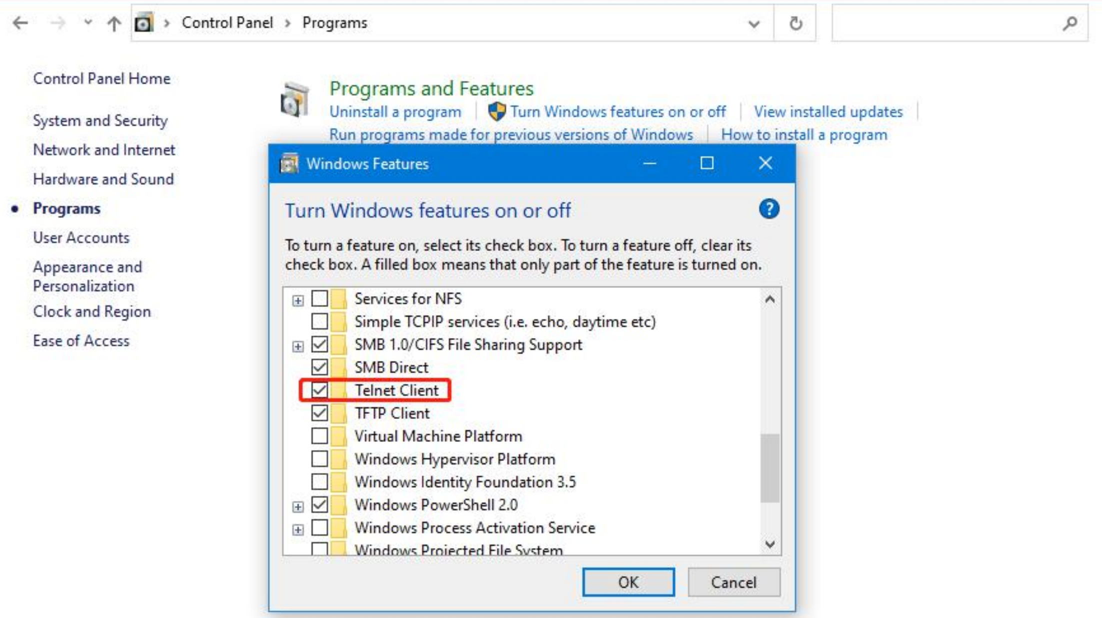
1.1.1 Setting IP Address in Your Computer

The detailed operation steps are omitted here, please consult your IT manager or search online for relevant guidance

1.1.2 Enabling Telnet Client

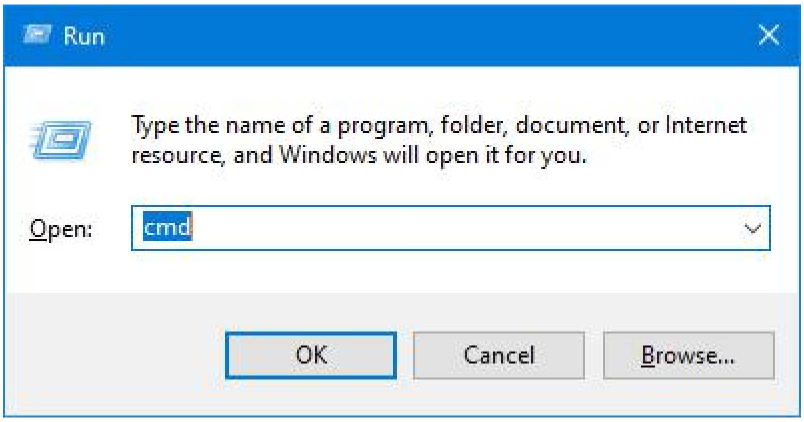
Before logging in to the device via command-line interface, make sure that **Telnet Client** is enabled. By default, **Telnet Client** is disabled in Windows OS. To turn on **Telnet Client**, do as follows.

1. Choose **Start** > **Windows System** > **Control Panel**.
2. In the popped dialog, click **Programs**.
3. In the **Programs and Features** area, click **Turn Windows features on or off**.
4. In **Windows Features** dialog box, select **Telnet Client** check box.



1.2 Logging In via Command Line Interface

1. Choose **Start** > **Run**.
2. In the **Run** dialog box, enter **cmd** then click **OK**.



3. Enter **telnet 169.254.1.100** if the device's IP address is 169.254.1.100, and then press **Enter**.

4. The device will show the command prompt as below:

Now, the device are ready to execute the CLI API command.

1.3 Access API via RS232 Interface

In addition to Telnet protocol, the API can be accessed via RS232 interface too. Here are brief instructions:

1. As the default, the RS232 interface is used to control the peripherals. To change its purpose to provide API services, you should invoke the API `SET UART_MODE` at first, the corresponding API request message is as follows

```
SET UART_MODE API
```

After completing the above operation, please reboot the device, then you can access the API via the RS232 interface.

2. The two special characters: Carriage Return and Line Breaks must be added at the end of each API request message in order. Their ASCII codes are 0x0D and 0x0A respectively. If you use a terminal emulation software (such as SecureCRT), the software will add these two characters automatically or else you should add these two characters manually.

1.4 Introduction

1.4.1 Terminology

The terminology used in API command description is listed as follows.

Terminology	Description
Device **Product**	The CAMSW100-000 unit being controlled. Unless otherwise specified, these two words appearing in this document refer to CAMSW100-000.
Hardware Source **Wired Source**	The video source which has dedicated hardware input interface, namely, the USB and HDMI sources, they are named **hardware (video) sources** or **wired sources**.
Network Source **NDI Source**	NDI video source has not dedicated hardware interface, all data is transported over the Ethernet interface, it is named **network source** or **NDI source**.

1.4.2 Classification

All APIs can be divided into the following categories according to their functions:

1. Device management

Manage the device name, reboot or reset device, etc...

2. Input & output management

Manage HDCP, output resolution, etc...

3. Video switching

Many different approaches to switch video source

4. Audio processing

Manage audio signal, such as volume, mute and routing

5. Standby related & peripheral control

Control the device to standby and wake-up, send CEC and RS232 message

6. Network

Manage IP address and network protocols

7. NDI management

Manage NDI related features

1.5 API Syntax Overview

1.5.1 Case-Handling Rules

1. For fields that are inherently case-sensitive—such as names and passwords—preserve the exact characters the user provided in the request and return them unchanged in the corresponding response. Fields currently covered by this rule are:

✧ *DeviceName* device name

✧ *CmdStr* RS232 port command string in text form

✧ NDI-related group, device, and channel names

2. For standardized units (e.g., dB, Hz), ignore case on input and always return the canonical form.
3. Internet domain names must be treated case-insensitively when processing requests and must always be returned in lowercase in responses.
4. For all other fields not mentioned above, treat the input case-insensitively and always return the value in UPPERCASE in the response.

1.5.2 Proactive Notification Mechanism

Any change that occurs on the device must be reported to the control clients immediately via unsolicited messages, following these rules:

1. The response message of a `GET` API is sent only to the control client that issued the `GET` request.
2. If a `SET` API fails to execute due to an error, only the control client that issued the request shall receive the corresponding error response.
3. If a `SET` API executes successfully, the response is broadcast to every currently connected control client as a real-time notification.
4. Whenever the operating state changes on its own (i.e., not triggered by a `SET` API), the device must proactively send the corresponding `SET`/`GET` API response to all currently connected control clients.

1.5.3 Terms & Conventions

1. The italic string in the message format indicates that the string should be replaced by the actual parameter in the actual message, and the italic string itself should not appear in the final message body. For example

```
DeviceName
```

Indicates the device name. The actual device name should be given in the actual message body.

2. Square brackets indicate optional parameters, for example

```
[ WinNo ]
```

indicates an optional window number parameter. Obviously, optional parameters can only appear at the end of the message.

3. The combination of curly braces and vertical bars is used to specify the possible values for the parameter, for example

```
< IN1 | IN2 | IN3 | IN4 >
```

Represents a parameter whose value can be one of `IN1`～`IN4`.

4. The combination of square brackets and vertical bars is used to specify the possible values for optional parameters, for example

```
[ START | STOP ]
```

Indicates an optional parameter whose value can be `START` or `STOP`

5. Definition of common error response messages:

```
ERROR ErrNo ErrReqMsg
```

If any error occurs when executing the API, the device can print specific prompt information. The response message begins with the word 'ERROR', following with an error number and the copy of the request message itself. The below screenshot is an example:

```
GET NULL
ERROR 1 : GET NULL
```

6. Some commonly used parameters

✧ Wired source

```
WiredSrc
```

Valid values are `IN1`, `IN2`, `IN3` and `IN4`, represents one of four wired video sources **USB IN1**, **USB IN2**, **HDMI IN1** and **HDMI IN2**.

✧ NDI source

```
NdiSrc
```

Valid values are `NDI1`, `NDI2`, `NDI3` and `NDI4`, indicating one of four NDI video sources.

✧ All input source

```
Src
```

Valid values are `IN1`, `IN2`, `IN3`, `IN4`, `NDI1`, `NDI2`, `NDI3` and `NDI4`， represents one of four wired sources and four NDI sources.

✧ DSP audio input

```
DspIn
```

One input channel or signal of the audio DSP, valid values are `USBIN`， `HDMIIN`, `USBHOST` and `ANALOGIN`，corresponding to the audio signal from the interface **USB IN1** (or **USB IN2**), **HDMI IN1** (or **HDMI IN2**), **USB HOST** and **AUDIO IN** (analog audio input).

✧ DSP audio output

```
DspOut
```

One output channel or signal of the audio DSP, valid values are `USBIN`， `HDMIOUT`, `USBHOST` and `ANALOGOUT`， corresponding to the audio signal sent to the interface **USB IN1** (and **USB IN2**), **HDMI OUT1** (and **HDMI IN2**), **USB HOST** and **AUDIO OUT** (analog audio output).

2 Command Details

2.1 Device management

2.1.1 Set device alias

【Format】

Request	`SET ALIAS` < *DeviceName* >
Response	`ALIAS` < *DeviceName* >
Description	Configure the device's alias. As a prompt, the new alias will appear on the top-right corner of the screen if the operation is successful. As the factory default, the alias is `CAMSW100-`*xxxxxx*, *xxxxxx* is the hexadecimal format of the last three digits of the device MAC address. **Note:** The device alias must be 1~31 characters in length, furthermore, it must include only letters, numbers, space and two special character ('_' and '-'). The beginning or end character must be a letter or number.

【Example】

To change the name to MeetingRoom:

Request:

```
SET ALIAS MeetingRoom
```

Response:

```
ALIAS MeetingRoom
```

2.1.2 Get device alias

【Format】

Request	`GET ALIAS`
Response	`ALIAS` < *DeviceName* >
Description	Obtain the device alias.

【Example】

Request:

```
GET ALIAS
```

Response:

```
ALIAS MeetingRoom
```

2.1.3 Set IP address

【Format】

Request	`SET IPADDR DHCP` Or `SET IPADDR STATIC` < *IPAddress* > < *NetMask* > [*Gateway* [*DNSServer*]]
Response	`IPADDR DHCP` Or `IPADDR STATIC` < *IPAddress* > < *NetMask* > [*Gateway* [*DNSServer*]]
Description	Configure the IP address, the two formats correspond to DHCP and static modes respectively. *Gateway* and *DNSServer* are optional parameters As the factory default, `DHCP` is used, if DHCP fails, the IP address `169.254.1.100` is used.

【Example】

Request:

```
SET IPADDR STATIC 192.168.1.34 255.255.255.0
```

Response:

```
IPADDR STATIC 192.168.1.34 255.255.255.0
```

2.1.4 Get IP address

【Format】

<table>
<tr><td>Request</td><td><code>GET IPADDR</code></td></tr>
<tr><td>Response</td><td><code>IPADDR < DHCP | STATIC > < IPAddress > < NetMask > [Gateway [DNSServer]]</code></td></tr>
<tr><td>Description</td><td>Query the IP address.</td></tr>
</table>

【Example】

Request:

```
GET IPADDR
```

Response:

```
IPADDR STATIC 192.168.1.34 255.255.255.0
```

2.1.5 Get firmware version

【Format】

<table>
<tr><td>Request</td><td><code>GET VER [ALL | ModuleName]</code></td></tr>
<tr><td>Response</td><td><code>VER ModuleName Version</code>
or
<code>VER ModuleName1 Version1</code>
<code>VER ModuleName2 Version2</code>
<code>...</code></td></tr>
<tr><td>Description</td><td>Obtain the firmware version information. The device returns the version information of the module designated by the parameter <code>ModuleName</code>, the available values are <code>MAINSOC</code>, <code>CAMCHIP</code> and <code>VIDEOCHIP</code>.
If the parameter value is <code>ALL</code>, the device responds with all modules' version information row by row.</td></tr>
</table>

【Example】

Request:

```
GET VER MAINSOC
```

Response:

```
VER MAINSOC 1.1.7
```

2.1.6 Get MAC address

【Format】

<table>
<tr><td>Request</td><td><code>GET MACADDR</code></td></tr>
<tr><td>Response</td><td><code>MACADDR MacAddress</code></td></tr>
<tr><td>Description</td><td>Obtain the MAC address, the response message prints the MAC address with colon separated form.</td></tr>
</table>

【Example】

Request:

```
GET MACADDR
```

Response:

```
MACADDR AA:BB:CC:DD:EE:07
```

2.1.7 System reboot

【Format】

Request	`REBOOT`
Response	`REBOOT`
Description	Reboot the device manually

【Example】

Request:

```
REBOOT
```

Response:

```
REBOOT
```

2.1.8 Factory reset

【Format】

Request	`RESET`
Response	`RESET`
Description	This command make the device restore its factory default, to archive this, the device will reboot to enter recovery or safe mode.

【Example】

Request:

```
RESET
```

Response:

```
RESET
```

2.2 Input & output settings

2.2.1 Set output resolution

【Format】

<table>
<tr><td>Request</td><td><code>SET VIDOUT_RES < OUT1 | OUT2 > < AUTO | Resolution ></code></td></tr>
<tr><td>Response</td><td><code>VIDOUT_RES < OUT1 | OUT2 > < AUTO | Resolution ></code></td></tr>
<tr><td>Description</td><td>Instruct the device to change its output resolution as the command designates or automatically.
The first parameter indicates which output resolution is set. If you assign <code>AUTO</code> as the <code>Resolution</code> parameter, the device will select a best resolution according to the display's EDID. The list of all available timings is below:
<code>3840x2160P@60* 3840x2160P@50* 3840x2160P@30 3840x2160P@25*</code>
<code>3840x2160P@24* 2560x1440P@60* 2560x1440P@30 1920x1080P@60</code></td></tr>
</table>

<table>
<tr><td></td><td>1920x1080P@50 1920x1080P@30 1680x1050P@60 1600x1200P@60
1440x900P@60 1366x768P@60 1280x1024P@60 1280x800P@60
1280x720P@60 1024x768P@60 720x576P@50 720x480P@60
*The resolution available for OUT1 only
As the factory default, both two outputs’ resolutions are set AUTO.</td></tr>
</table>

【Example】

To use 4K@60 resolution on the output 1:

Request:

```
SET VIDOUT_RES OUT1 3840x2160P@60
```

Response:

```
VIDOUT_RES OUT1 3840x2160P@60
```

2.2.2 Get output resolution

【Format】

<table>
<tr><td>Request</td><td>GET VIDOUT_RES < OUT1 | OUT2 | USB | ALL ></td></tr>
<tr><td>Response</td><td>VIDOUT_RES < OUT1 | OUT2 | USB > < Resolution | DISCONNECTED > [AUTO]

or
VIDOUT_RES OUT1 < Resolution | DISCONNECTED > [AUTO]
VIDOUT_RES OUT2 < Resolution | DISCONNECTED > [AUTO]
VIDOUT_RES USB < Resolution | DISCONNECTED ></td></tr>
<tr><td>Description</td><td>Get the output resolution information of the designated output. If the parameter value is ALL, the device prints the resolution information of two outputs in two rows.
For each row, the first field is the number of the output, the second field is the actual resolution. The third field AUTO is optional which means the corresponding output resolution is set AUTO.
The term DISCONNECTED have two different meanings:
✧ The resolution of the HDMI output interface is set AUTO but the output is not connected to a display.
✧ The UVC device of the USB output interface is not opened by the host computer.</td></tr>
</table>

【Example】

Request:

```
GET VIDOUT_RES ALL
```

Response:

```
VIDOUT_RES OUT1 3840x2160@60 AUTO
VIDOUT_RES OUT2 DISCONNECTED AUTO
VIDOUT_RES USB DISCONNECTED
```

2.2.3 Get output connection status

【Format】

<table>
<tr><td>Request</td><td>GET VIDOUT_CONNECT < OUT1 | OUT2 | USB | ALL ></td></tr>
<tr><td>Response</td><td>VIDOUT_CONNECT < OUT1 | OUT2 | USB > < DISCONNECTED | CONNECTED >
or
VIDOUT_CONNECT OUT1 < DISCONNECTED | CONNECTED ></td></tr>
</table>

	VIDOUT_CONNECT OUT2 < DISCONNECTED | CONNECTED > VIDOUT_CONNECT USB < DISCONNECTED | CONNECTED >
Description	If the parameter designates a specific video output, it means to query whether the specific output is connected to a source, the device prints CONNECTED to indicate the source is connected, and vice versa. if the parameter value is ALL, the device responds with three rows of texts to report the connection status of all video outputs.

【Example】

Request:

```
GET VIDOUT_CONNECT USB
```

Response:

```
VIDOUT_CONNECT USB CONNECTED
```

2.2.4 Set HD mode

【Format】

Request	SET UVC_HDMODE < ON | OFF >
Response	UVC_HDMODE < ON | OFF >
Description	Turn on/off the HD mode of the USB output. USB HOST interface provide video output function with UVC specification, the output resolution can not be designated by the device itself, on the contrary, what the device can do is to provide a list of all supported resolutions, the host computer chooses one from the list. This API can modify the content of the aforementioned list, when the configuration is ON, the list contains the resolutions equals to 720P@30 or higher. As the factory default, HD mode is enabled, the corresponding configuration is ON.

【Example】

Request:

```
SET UVC_HDMODE OFF
```

Response:

```
UVC_HDMODE OFF
```

2.2.5 Get HD mode

【Format】

Request	GET UVC_HDMODE
Response	UVC_HDMODE < ON | OFF >
Description	Get the HD mode configuration.

【Example】

Request:

```
GET UVC_HDMODE
```

Response:

```
UVC_HDMODE OFF
```

2.2.6 Set output HDCP configuration

【Format】

<table>
<tr><td>Request</td><td><code>SET HDCP < OUT1 | OUT2 > < ON | OFF ></code></td></tr>
<tr><td>Response</td><td><code>HDCP < OUT1 | OUT2 > < ON | OFF ></code></td></tr>
<tr><td>Description</td><td>Set the HDCP feature of a output.
As the factory default, HDCP feature is enabled, the corresponding configuration is <code>ON</code>.</td></tr>
</table>

【Example】

Request:

```
SET HDCP OUT1 OFF
```

Response:

```
HDCP OUT1 OFF
```

2.2.7 Get output HDCP configuration

【Format】

<table>
<tr><td>Request</td><td><code>GET HDCP < OUT1 | OUT2 | ALL ></code></td></tr>
<tr><td>Response</td><td><code>HDCP < OUT1 | OUT2 > < ON | OFF ></code>
or
<code>HDCP OUT1 < ON | OFF ></code>
<code>HDCP OUT2 < ON | OFF ></code></td></tr>
<tr><td>Description</td><td>Get the HDCP configuration of the designated output. If the parameter value is <code>ALL</code>, the device prints HDCP configuration information of two outputs.</td></tr>
</table>

【Example】

Request:

```
GET HDCP OUT1
```

Response:

```
HDCP OUT1 OFF
```

2.2.8 Get input connection status

【Format】

<table>
<tr><td>Request</td><td><code>GET VIDIN_CONNECT < Src | ALL ></code></td></tr>
<tr><td>Response</td><td><code>VIDIN_CONNECT < Src > < DISCONNECTED | CONNECTED ></code>
or
<code>VIDIN_CONNECT IN1 < DISCONNECTED | CONNECTED ></code>
<code>VIDIN_CONNECT IN2 < DISCONNECTED | CONNECTED ></code>
<code>VIDIN_CONNECT IN3 < DISCONNECTED | CONNECTED ></code>
<code>VIDIN_CONNECT IN4 < DISCONNECTED | CONNECTED ></code>
<code>VIDIN_CONNECT NDI1 < DISCONNECTED | CONNECTED ></code>
<code>VIDIN_CONNECT NDI2 < DISCONNECTED | CONNECTED ></code>
<code>VIDIN_CONNECT NDI3 < DISCONNECTED | CONNECTED ></code>
<code>VIDIN_CONNECT NDI4 < DISCONNECTED | CONNECTED ></code></td></tr>
<tr><td>Description</td><td>If the parameter designates a specific video source, it means to query whether the specific input is connected to a source, the device prints</td></tr>
</table>

<table>
<tr><td></td><td>CONNECTED to indicate the source is connected, and vice versa.
if the parameter value is ALL, the device responds with eight rows of texts to report the connection status of all video sources.</td></tr>
</table>

【Example】

Request:

```
GET VIDIN_CONNECT IN2
```

Response:

```
VIDIN_CONNECT IN2 DISCONNECTED
```

2.2.9 Get input signal status

【Format】

<table>
<tr><td>Request</td><td>GET VIDIN_SIG < Src | ALL ></td></tr>
<tr><td>Response</td><td>VIDIN_SIG < Src > < NO | VALID >
or
VIDIN_SIG IN1 < NO | VALID >
VIDIN_SIG IN2 < NO | VALID >
VIDIN_SIG IN3 < NO | VALID >
VIDIN_SIG IN4 < NO | VALID >
VIDIN_SIG NDI1 < NO | VALID >
VIDIN_SIG NDI2 < NO | VALID >
VIDIN_SIG NDI3 < NO | VALID >
VIDIN_SIG NDI4 < NO | VALID ></td></tr>
<tr><td>Description</td><td>If the parameter designates a specific source, it means to query whether the specific source has valid signal, the device prints VALID to indicate the source has valid signal, and vice versa.
if the parameter value is ALL, the device responds with eight rows of texts to report the signal status of all video sources.</td></tr>
</table>

【Example】

Request:

```
GET VIDIN_SIG IN3
```

Response:

```
VIDIN_SIG IN3 VALID
```

2.2.10 Get input video format

【Format】

<table>
<tr><td>Request</td><td>GET VIDIN_FORMAT < Src | ALL ></td></tr>
<tr><td>Response</td><td>VIDIN_FORMAT < Src | ALL > < INVALID | VideoFormat >
or
VIDIN_FORMAT IN1 < INVALID | VideoFormat >
VIDIN_FORMAT IN2 < INVALID | VideoFormat >
VIDIN_FORMAT IN3 < INVALID | VideoFormat >
VIDIN_FORMAT IN4 < INVALID | VideoFormat >
VIDIN_FORMAT NDI1 < INVALID | VideoFormat >
VIDIN_FORMAT NDI2 < INVALID | VideoFormat >
VIDIN_FORMAT NDI3 < INVALID | VideoFormat >
VIDIN_FORMAT NDI4 < INVALID | VideoFormat ></td></tr>
<tr><td>Description</td><td>Query the video format of the designated input, the device prints INVALID to indicate the source has not valid signal, or else the device prints the VideoFormat whose format is:</td></tr>
</table>

	`< Width x Height[p],FrameRate;HDRInfo;ColorSpace;ColorDepth >`
	The value and meaning of every field are as follows:
	✧ *Width*, *Height* The width and height of the video. In the actual message, there is no space character between the Width, height and the letter 'X', for an example, 1920X1080
	✧ P It is optional to indicate progressive video
	✧ *FrameRate* Frame rate
	✧ *HDRInfo* Whether HDR format, now the device does not support HDR, so its value is always NONE HDR.
	✧ *ColorSpace* Color space, available values are RGB、YCBCR444、YCBCR422
	✧ *ColorDepth* Color depth, *now only* 8BIT
	An actual example of *VideoFormat*:
	`3840X2160P,30;NONE HDR;YCBCR444;8BIT`
	if the request message parameter is ALL, the device prints the video format information of all video sources.

【Example】

Request:

```
GET VIDIN_FORMAT IN3
```

Response:

```
VIDIN_FORMAT IN3 3840X2160P,30;NONE HDR;YCBCR444;8BIT
```

2.2.11 Get input audio format

【Format】

Request	`GET AUDIN_FORMAT < Src | ALL >`

Response	`AUDIN_FORMAT < Src | ALL > < INVALID | AudioFormat >`
	or
	`AUDIN_FORMAT IN1 < INVALID | AudioFormat >`
	`AUDIN_FORMAT IN2 < INVALID | AudioFormat >`
	`AUDIN_FORMAT IN3 < INVALID | AudioFormat >`
	`AUDIN_FORMAT IN4 < INVALID | AudioFormat >`
	`AUDIN_FORMAT NDI1 < INVALID | AudioFormat >`
	`AUDIN_FORMAT NDI2 < INVALID | AudioFormat >`
	`AUDIN_FORMAT NDI3 < INVALID | AudioFormat >`
	`AUDIN_FORMAT NDI4 < INVALID | AudioFormat >`

Description	Query the audio format of the designated input, the device prints INVALID to indicate the source has not valid audio signal, or else the device prints the *AudioFormat* whose format is:
	`< Encoding;SamplingRate;SampingWidth >`
	The value and meaning of every field are as follows:
	✧ *Encoding* The audio encoding mode, the available values are PCM, AAC and MP3
	✧ *SamplingRate* Sampling rate, such as 48KHz
	✧ *SampingWidth* Sampling width, the available values are 8BIT and 16BIT.
	An actual example of *AudioFormat*:
	`PCM;48KHz;8BIT`
	if the request message parameter is ALL, the device prints the audio format information of all video sources.

【Example】

Request:

```
GET AUDIN_FORMAT IN3
```

Response:

```
AUDIN_FORMAT IN3 PCM;48KHz;8BIT
```

2.3 Video switching

2.3.1 Set low latency mode

【Format】

Request	`SET LOW_LATENCY_FN < ON | OFF >`
Response	`LOW_LATENCY_FN < ON | OFF >`
Description	Turn on/off the low latency mode. When this feature is enabled, the device uses 1920x1080 resolution to procession video signals whenever possible, so to achieve as low latency as possible. Here is the list of the items whose resolution affected: ✧ Opening USB input ✧ USB output ✧ HDMI output ✧ NDI output (Primary stream) Furthermore, when low latency mode is enabled, the video switching is faster. To employee the higher resolution, please disable this feature. As the factory default, low latency mode is enabled, the corresponding configuration is `ON`.

【Example】

Request:

```
SET LOW_LATENCY_FN OFF
```

Response:

```
LOW_LATENCY_FN OFF
```

2.3.2 Get low latency mode

【Format】

Request	`GET LOW_LATENCY_FN`
Response	`LOW_LATENCY_FN < ON | OFF >`
Description	Get the low latency mode configuration.

【Example】

Request:

```
GET LOW_LATENCY_FN
```

Response:

```
LOW_LATENCY_FN OFF
```

2.3.3 Set input to output

【Format】

<table>
<tr><td>Request</td><td><code>SET SW < Src | NONE > [OUT1 | OUT2]</code></td></tr>
<tr><td>Response</td><td><code>SW < Src | NONE > [OUT1 | OUT2]</code></td></tr>
<tr><td>Description</td><td>Switch an input to an output, this command instructs the device to display a video source on the specified video output. The specific behavior is as follows:
✧ The device displays the video source with full screen mode always, no matter whether or which multiview layout is used currently.
✧ If the request message does not carry the optional parameter, the device display the video source in full screen on all outputs.
✧ If the video source to be displayed is <code>NONE</code>, the device stops displaying video source the guide screen.</td></tr>
</table>

【Example】

Request:

```
SET SW IN3 OUT1
```

Response:

```
SW IN3 OUT1
```

2.3.4 Get input of output

【Format】

<table>
<tr><td>Request</td><td><code>GET SW < OUT1 | OUT2 | ALL ></code></td></tr>
<tr><td>Response</td><td><code>SW < Src | NONE | MV > < OUT1 | OUT2 ></code>
or
<code>SW < Src | NONE > OUT1</code>
<code>SW < Src | NONE | MV > OUT2</code></td></tr>
<tr><td>Description</td><td>Get the information which video source is being displayed on an output. When the request message uses <code>ALL</code> as the parameter, the device responds with two rows to provide the status of two video output signals (<code>USB</code> output has the same video content with <code>OUT2</code>).
In the response message, <code>NONE</code> means no video source is shown on the output, <code>MV</code> means <code>OUT2</code> is in multiview state.</td></tr>
</table>

【Example】

Request:

```
GET SW OUT1
```

Response:

```
SW IN3 OUT1
```

2.3.5 Set multiview layout

【Format】

<table>
<tr><td>Request</td><td><code>SET MV_LAYOUT < LayoutID | LayoutName ></code></td></tr>
<tr><td>Response</td><td><code>MV_LAYOUT < LayoutID | LayoutName ></code></td></tr>
</table>

Description	Switch the layout used by multiview mode, multiview feature works on `OUT2` only, the parameter *`LayoutID`* or *`LayoutName`* is used to specify a layout. The IDs and corresponding names of all supported layouts are listed below: ✧ `0x100` `FullScreen` ✧ `0x101` `PiP` ✧ `0x102` `SbS` ✧ `0x103` `LBRS` ✧ `0x104` `Pyramid` ✧ `0x105` `Quad` As the boot default, single full screen layout is used, the corresponding *`LayoutID`* is `0x100` and the *`LayoutName`* is `FullScreen`.

【Example】

Request:

```
SET MV_LAYOUT QUAD
```

Response:

```
MV_LAYOUT QUAD
```

2.3.6 Get multiview layout

【Format】

Request	`GET MV_LAYOUT`
Response	`MV_LAYOUT` *`LayoutID LayoutName`*
Description	Get the multiview layout used currently, the device returns the ID and name of the layout.

【Example】

Request:

```
GET MV_LAYOUT
```

Response:

```
MV_LAYOUT 0X101 PIP
```

2.3.7 Set multiview layout and video source(s)

【Format】

Request	`SET MV_WIN_SRC <` *`LayoutID`* `|` *`LayoutName`* `| Current >` *`WinNo1`* `<` *`Src1`* `| NONE > [` *`WinNo2`* `<` *`Src2`* `| NONE > ] ...`
Response	`MV_WIN_SRC <` *`LayoutID`* `|` *`LayoutName`* `>` *`WinNo1`* `<` *`Src1`* `| NONE > [` *`WinNo2`* `<` *`Src2`* `| NONE > ] ...`
Description	Switch the layout and set video source(s) for the child window(s). The parameter *WinNo* can use the one of the following two forms: ✧ Number Such as `1, 2, 3...` ✧ `Win1, Win2, Win3...` The parameter value `NONE` means no video source is shown in the corresponding window. If first parameter value of the request message is `Current`, it means to remain the current layout unchanged.

	If the designated layout is the same as the current layout, the device change the video source(s) of the window(s) appearing in the parameters only, the rest window(s) will not be impact. In short, this usage equivalent to switching the video source(s) of the specified child window(s).

【Example】

Request:

```
SET MV_WIN_SRC QUAD 1 IN2
```

Response:

```
MV_WIN_SRC QUAD 1 IN2
```

2.3.8 Get multiview layout and video source(s)

【Format】

Request	GET MV_WIN_SRC
Response	MMV_WIN_SRC *LayoutName WinNo1* < *Src1* | NONE > [*WinNo2* < *Src2* | NONE >] ...
Description	Get the multiview layout and video source(s) of the child window(s).

【Example】

Request:

```
GET MV_WIN_SRC
```

Response:

```
MV_WIN_SRC SbS WIN1 IN2 WIN2 NONE
```

2.4 Audio processing

2.4.1 Set DSP audio mixing

【Format】

Request	SET DSP_MIX < *DspOut* > < *DspIn* > < ON | OFF >
Response	DSP_MIX < *DspOut* > < *DspIn* > < ON | OFF >
Description	The device has a built-in audio DSP to support auido matrix and mixer functions. It supports 4 input channels (*DspIn*: USBIN, HDMIIN, USBHOST, ANALOGIN) and 4 output channels (*DspOut*: USBIN, HDMIOUT, USBHOST, ANALOGOUT). The audio mixing between any input channel and any output channel is achieved. This API is used to set whether the specified audio input channel participates in the mixing of the specified audio output channel. To avoid audio loop or self-excitation, the following mixing relationships are prohibited: ✧ USBIN --> USBIN ✧ USBHOST --> USBHOST ✧ ANALOGIN --> ANALOGOUT As the factory default, the USBHOST channel participates the mixing of the

<table>
<tr><td></td><td>USBIN channel, the mixing configuration of the corresponding crosspoint is ON, for the remaining crosspoints, the configurations are both OFF.</td></tr>
</table>

【Example】

Request:

```
SET DSP_MIX HDMIOUT HDMIIN ON
```

Response:

```
DSP_MIX HDMIOUT HDMIIN ON
```

2.4.2 Get DSP audio mixing

【Format 1】

<table>
<tr><td>Request</td><td>GET DSP_MIX < DspOut > < DspIn ></td></tr>
<tr><td>Response</td><td>DSP_MIX < DspOut > < DspIn > < ON | OFF ></td></tr>
</table>

【Format 2】

<table>
<tr><td>Request</td><td>GET DSP_MIX < DspOut | ALL ></td></tr>
<tr><td>Response</td><td>DSP_MIX < DspOut > [< DspIn1 > [DspIn2] ...]
or
DSP_MIX USBIN [< DspIn1 > [DspIn2] ...]
DSP_MIX HDMIOUT [< DspIn1 > [DspIn2] ...]
DSP_MIX USBHOST [< DspIn1 > [DspIn2] ...]
DSP_MIX ANALOGOUT [< DspIn1 > [DspIn2] ...]</td></tr>
</table>

【Comment】

<table>
<tr><td>Description</td><td>The format 1 is used to get the mixing configuration of the specified audio input channel and the specified audio output channel.
The format 2 can get the mixing configuration of the specified audio output channel, the response message lists all audio input channels which participate the audio mixing of the specified audio output channel. If he request message parameter value is ALL, it means to get the mixing configuration of all audio output channels.</td></tr>
</table>

【Example 1】

Request:

```
GET DSP_MIX USBIN USBHOST
```

Response:

```
DSP_MIX USBIN USBHOST ON
```

【Example 2】

Request:

```
GET DSP_MIX ALL
```

Response:

```
DSP_MIX USBIN USBHOST
DSP_MIX HDMIOUT HDMIIN
DSP_MIX USBHOST
DSP_MIX ANALOGOUT HDMIIN
```

2.4.3 Set DSP input channel

【Format】

<table>
<tr><td>Request</td><td>SET DSP_AUD_SRC < < USBIN < IN1 | IN2 > > | < HDMIIN < IN3 | IN4 > > ></td></tr>
<tr><td>Response</td><td>DSP_AUD_SRC < < USBIN < IN1 | IN2 > > | < HDMIIN < IN3 | IN4 > > ></td></tr>
</table>

Description	Only one audio signal of the two USB inputs can be sent to the audio DSP for processing, the two HDMI inputs have the same limitation too. This API chooses the one audio signal (sent to the audio DSP) from the two USB or HDMI inputs, or in another word, the chosen audio signal will be sent to the `USBIN` or `HDMIIN` channel of the audio DSP. As the factory default, `IN1` and `IN3` are chosen for `USBIN` and `HDMIIN` respectively.

【Example】

Request:

```
SET DSP_AUD_SRC USBIN IN1
```

Response:

```
DSP_AUD_SRC USBIN IN1
```

2.4.4 Get DSP input channel

【Format】

Request	`GET DSP_AUD_SRC < USBIN | HDMIIN | ALL >`

Response	`DSP_AUD_SRC < < USBIN < IN1 | IN2 > > | < HDMIIN < IN3 | IN4 > > >`
	or
	`DSP_AUD_SRC USBIN < IN1 | IN2 >`
	`DSP_AUD_SRC HDMIIN < IN3 | IN4 >`

Description	If he request message parameter value is `ALL`, it means to get the input channel configurations of `USBIN` and `HDMIIN` together.

【Example】

Request:

```
GET DSP_AUD_SRC USBIN
```

Response:

```
DSP_AUD_SRC USBIN IN1
```

2.4.5 Set DSP input mute

【Format】

Request	`SET DSP_AUD_GAIN_MUTE <` *`DspIn`* `> < ON | OFF >`
Response	`DSP_AUD_GAIN_MUTE <` *`DspIn`* `> < ON | OFF >`
Description	Set whether the specified audio input channel is muted. As the factory default, all channels are not mute, the configuration value is `OFF`.

【Example】

Request:

```
SET DSP_AUD_GAIN_MUTE USBIN ON
```

Response:

```
DSP_AUD_GAIN_MUTE USBIN ON
```

2.4.6 Get DSP input mute

【Format】

<table>
<tr><td>Request</td><td><code>GET DSP_AUD_GAIN_MUTE < DspIn | ALL ></code></td></tr>
<tr><td>Response</td><td><code>DSP_AUD_GAIN_MUTE < DspIn > < ON | OFF ></code>
or
<code>DSP_AUD_GAIN_MUTE USBIN < ON | OFF ></code>
<code>DSP_AUD_GAIN_MUTE HDMIIN < ON | OFF ></code>
<code>DSP_AUD_GAIN_MUTE USBHOST < ON | OFF ></code>
<code>DSP_AUD_GAIN_MUTE ANALOGIN < ON | OFF ></code></td></tr>
<tr><td>Description</td><td>If the request message parameter value is <code>ALL</code>, it means to get the mute configurations of all input channels.</td></tr>
</table>

【Example】

Request:

```
GET DSP_AUD_GAIN_MUTE USBIN
```

Response:

```
DSP_AUD_GAIN_MUTE USBIN ON
```

2.4.7 Set DSP input gain

【Format】

<table>
<tr><td>Request</td><td><code>SET DSP_AUD_GAIN < DspIn > < Gain ></code></td></tr>
<tr><td>Response</td><td><code>DSP_AUD_GAIN < DspIn > < Gain ></code></td></tr>
<tr><td>Description</td><td>The unit of the parameter <code>Gain</code> is dB and the available range is <code>[-20, +30]</code>, when the device processes this API request, the mute configuration of the corresponding input channel is disabled (<code>OFF</code>) too.
As the factory default, the gain is not adjusted, namely, <code>Gain</code> is <code>0</code>.</td></tr>
</table>

【Example】

Request:

```
SET DSP_AUD_GAIN USBIN 3
```

Response:

```
DSP_AUD_GAIN USBIN 3
```

2.4.8 Get DSP input gain

【Format】

<table>
<tr><td>Request</td><td><code>GET DSP_AUD_GAIN < DspIn | ALL ></code></td></tr>
<tr><td>Response</td><td><code>DSP_AUD_GAIN < DspIn > < Gain ></code>
or
<code>DSP_AUD_GAIN USBIN < Gain1 ></code>
<code>DSP_AUD_GAIN HDMIIN < Gain2 ></code>
<code>DSP_AUD_GAIN USBHOST < Gain3 ></code>
<code>DSP_AUD_GAIN ANALOGIN < Gain4 ></code></td></tr>
<tr><td>Description</td><td>If the request message parameter value is <code>ALL</code>, it means to get the gain configurations of all input channels.</td></tr>
</table>

【Example】

Request:

```
GET DSP_AUD_GAIN USBIN
```

Response:

```
DSP_AUD_GAIN USBIN 3
```

2.4.9 Set DSP output mute

【Format】

Request	`SET DSP_AUD_VOL_MUTE < DspOut > < ON | OFF >`
Response	`DSP_AUD_VOL_MUTE < DspOut > < ON | OFF >`
Description	Set whether the specified audio output channel is muted. As the factory default, all channels are not mute, the configuration value is `OFF`.

【Example】

Request:

```
SET DSP_AUD_VOL_MUTE USBHOST ON
```

Response:

```
DSP_AUD_VOL_MUTE USBHOST ON
```

2.4.10 Get DSP output mute

【Format】

Request	`GET DSP_AUD_VOL_MUTE < DspOut | ALL >`

Response	`DSP_AUD_VOL_MUTE < DspOut > < ON | OFF >`
	or
	`DSP_AUD_VOL_MUTE USBIN < ON | OFF >`
	`DSP_AUD_VOL_MUTE HDMIOUT < ON | OFF >`
	`DSP_AUD_VOL_MUTE USBHOST < ON | OFF >`
	`DSP_AUD_VOL_MUTE ANALOGOUT < ON | OFF >`

Description	If the request message parameter value is `ALL`, it means to get the mute configurations of all output channels.

【Example】

Request:

```
GET DSP_AUD_VOL_MUTE USBHOST
```

Response:

```
DSP_AUD_VOL_MUTE USBHOST ON
```

2.4.11 Set DSP output volume

【Format】

Request	`SET DSP_AUD_VOL < DspOut > < Volume >`
Response	`DSP_AUD_VOL < DspOut > < Volume>`
Description	The unit of the parameter *`Volume`* is dB and the available range is `[-100, 0]`, when the device processes this API request, the mute configuration of the corresponding output channel is disabled (`OFF`) too. As the factory default, the gain is not adjusted, namely, *`Volume`* is `0`.

【Example】

Request:

```
SET DSP_AUD_VOL HDMIOUT -6
```
Response:
```
DSP_AUD_VOL HDMIOUT -6
```

2.4.12 Get DSP output volume

【Format】

Request	`GET DSP_AUD_VOL < DspOut | ALL >`

Response	`DSP_AUD_VOL < DspOut > < Gain >`
	or
	`DSP_AUD_VOL USBIN < Gain1 >`
	`DSP_AUD_VOL HDMIOUT < Gain2 >`
	`DSP_AUD_VOL USBHOST < Gain3 >`
	`DSP_AUD_VOL ANALOGOUT < Gain4 >`

Description	If the request message parameter value is `ALL`, it means to get the volume configurations of all output channels.

【Example】

Request:
```
GET DSP_AUD_VOL HDMIOUT
```
Response:
```
DSP_AUD_VOL HDMIOUT -6
```

2.5 Standby related & peripheral control

2.5.1 Set standby mode

【Format】

Request	`SET STANDBY < ON | OFF >`
Response	`STANDBY < ON | OFF >`
Description	Manually instruct the device to enters or exits standby mode. `ON` means power on (exiting standby mode) and `OFF` means power off (entering standby mode).

【Example】

Request:
```
SET STANDBY OFF
```
Response:
```
STANDBY OFF
```

2.5.2 Get standby mode

【Format】

Request	`GET STANDBY`
Response	`STANDBY < ON | OFF >`
Description	Query whether the device is in standby mode.

【Example】

Request:
```
GET STANDBY
```
Response:
```
STANDBY OFF
```

2.5.3 Set automatic standby

【Format】

Request	`SET AUTO_STANDBY_FN < ON | OFF >`
Response	`AUTO_STANDBY_FN  < ON | OFF >`
Description	Set whether the automatic standby feature is enabled. As the factory default, automatic standby feature is enabled, the configuration value is `ON`.

【Example】

Request:
```
SET AUTO_STANDBY_FN OFF
```
Response:
```
AUTO_STANDBY_FN OFF
```

2.5.4 Get automatic standby

【Format】

Request	`GET AUTO_STANDBY_FN`
Response	`AUTO_STANDBY_FN  < ON | OFF >`
Description	Query the state of the automatic standby configuration

【Example】

Request:
```
GET AUTO_STANDBY_FN
```
Response:
```
AUTO_STANDBY_FN OFF
```

2.5.5 Set automatic standby timeout

【Format】

Request	`SET AUTO_STANDBY_D <` *`TimeOut`* `>`
Response	`AUTO_STANDBY_D <` *`TimeOut`* `>`
Description	The parameter *`Timeout`* indicates how long the device will be idle (namely, displaying guide screen) before entering standby mode, its unit is second. The value `0` means entering standby mode immediately when the device starts being idle. Apparently this configuration takes effect only when the automatic standby feature is enabled. The value range is from `0` to `3600`.

<table>
<tr><td></td><td>As the factory default, automatic standby timeout is 2 minutes, the configuration value is set 120.</td></tr>
</table>

【Example】

Request:

```
SET AUTO_STANDBY_D 360
```

Response:

```
AUTO_STANDBY_D 360
```

2.5.6 Get automatic standby timeout

【Format】

<table>
<tr><td>Request</td><td><code>GET AUTO_STANDBY_D</code></td></tr>
<tr><td>Response</td><td><code>AUTO_STANDBY_D < TimeOut ></code></td></tr>
<tr><td>Description</td><td>Query the value of the automatic standby timeout</td></tr>
</table>

【Example】

Request:

```
GET AUTO_STANDBY_D
```

Response:

```
AUTO_STANDBY_D 360
```

2.5.7 Set CEC command

【Format】

<table>
<tr><td>Request</td><td><code>SET CECCMD_EDIT < OUT1 | OUT2 > < ON | OFF > < CECCode ></code></td></tr>
<tr><td>Response</td><td><code>CECCMD_EDIT < OUT1 | OUT2 > < ON | OFF > < CECCode ></code></td></tr>
<tr><td>Description</td><td>Configure the stored CEC command used to power on/off the display.
The first parameter indicates which output to be set. The second parameter indicates which command will be changed. The third parameter CECCode is the actual CEC message string with hexadecimal format, no space between the adjacent bytes.
As the factory default, the power on (ON) message is 4004 and power off (OFF) message is FF36.</td></tr>
</table>

【Example】

Request:

```
SET CECCMD_EDIT OUT1 ON 4004
```

Response:

```
CECCMD_EDIT OUT1 ON 4004
```

2.5.8 Get CEC command

【Format】

<table>
<tr><td>Request</td><td><code>GET CECCMD_EDIT < < OUT1 | OUT2 > < ON | OFF > | ALL ></code></td></tr>
<tr><td>Response</td><td><code>CECCMD_EDIT < OUT1 | OUT2 > < ON | OFF > < CECCode ></code>
Or
<code>CECCMD_EDIT OUT1 ON < CECCode ></code></td></tr>
</table>

	CECCMD_EDIT OUT1 OFF < *CECCode* > CECCMD_EDIT OUT2 ON < *CECCode* > CECCMD_EDIT OUT2 OFF < *CECCode* >
Description	Query a CEC command configuration. If the request message parameter value is ALL, the device returns all configurations in four rows.

【Example】

Request:

```
GET CECCMD_EDIT OUT1 ON
```

Response:

```
CECCMD_EDIT OUT1 ON 4004
```

2.5.9 Set RS232 work mode

【Format】

Request	SET UART_MODE < API | COM >
Response	UART_MODE < API | COM >
Description	The RS232 port supports different work modes, the value and meanings of the parameter are as follows: ✧ API Receive the API commands sent by the external controller ✧ COM Conventional communication, control the peripherals such as display, participate in standby related power on/off display operations. As the factory default, the RS232 port is used to control the peripherals the configuration value is COM.

【Example】

Request:

```
SET UART_MODE API
```

Response:

```
UART_MODE API
```

2.5.10 Get RS232 work mode

【Format】

Request	GET UART_MODE
Response	UART_MODE < API | COM >
Description	Query the work mode of the RS232 port.

【Example】

Request:

```
GET UART_MODE
```

Response:

```
UART_MODE API
```

2.5.11 Set power on/off display via RS232

【Format】

Request	`SET UARTPWR_FN < ON | OFF >`
Response	`UARTPWR_FN < ON | OFF >`
Description	When the device enters or exits standby mode, in additions to powering on/off the display through CEC messages, the device also can send the corresponding commands through RS232 port. This API is used to configure whether to send RS232 command when powering on/off the display. **Note**: The actual activation of sending power on/off command through RS232 port requires three conditions to be met: ✧ The RS232 port work mode is `COM`, namely, control peripherals ✧ The RS232 `ON` or `OFF` message is configured ✧ This configuration is `ON` As the factory default, this configuration is set `OFF`, when the device enters or exits standby mode, the device sends power on/off command through CEC messages only.

【Example】

Request:

```
SET UARTPWR_FN ON
```

Response:

```
UARTPWR_FN ON
```

2.5.12 Get power on/off display via RS232

【Format】

Request	`GET UARTPWR_FN`
Response	`UARTPWR_FN < ON | OFF >`
Description	Query the configuration.

【Example】

Request:

```
GET UARTPWR_FN
```

Response:

```
UARTPWR_FN ON
```

2.5.13 Configure RS232 settings

【Format】

Request	`SET UART_CFG <` *`BaudRate`* `> <` *`Parity`* `> <` *`DataBit`* `> <` *`StopBit`* `>`
Response	`UART_CFG <` *`BaudRate`* `> <` *`Parity`* `> <` *`DataBit`* `> <` *`StopBit`* `>`
Description	Modify the RS232 port settings, the values and corresponding meanings of the parameters are below: ✧ *`BaudRate`* Baud rate, available values are `9600`, `19200`, `38400`,

	57600, 115200 ✧ *Parity* NONE, ODD, EVEN ✧ *DataBit* Number of data bits, 7 or 8 ✧ *StopBit* Number of stop bits, 1 or 2 As the factory default, the value of the above settings are 115200, NONE, 8, 1 respectively.

【Example】

Request:

```
SET UART_CFG 9600 NONE 8 1
```

Response:

```
UART_CFG 9600 NONE 8 1
```

2.5.14 Get RS232 setting

【Format】

Request	GET UART_CFG
Response	UART_CFG < *BaudRate* > < *Parity* > < *DataBit* > < *StopBit* >
Description	Query the settings of the RS232 port.

【Example】

Request:

```
GET UART_CFG
```

Response:

```
UART_CFG 9600 NONE 8 1
```

2.5.15 Set RS232 baud rate

【Format】

Request	SET UART_B < *BaudRate* >
Response	UART_B < *BaudRate* >
Description	Configure the RS232 port baud rate separately, the available values are 9600, 19200, 38400, 57600, 115200. As the factory default, the baud rate is 115200.

【Example】

Request:

```
SET UART_B 9600
```

Response:

```
UART_B 9600
```

2.5.16 Get RS232 baud rate

【Format】

Request	GET UART_B
Response	UART_B < *BaudRate* >
Description	Query the baud rate of the RS232 port.

【Example】

Request:
```
GET UART_B
```
Response:
```
UART_B 9600
```

2.5.17 Set RS232 command

【Format】

Request	`SET UART_CMD < CmdName > < HEX | STR > < CmdStr >`
Response	`UART_CMD < CmdName > < HEX | STR > < CmdStr >`

Description	Configure the message string sent to the peripherals through the RS232 port. The values and meanings of the parameters are as follows:
	1. The *CmdName* parameter indicates which command to be configured. Except `ON` and `OFF` (used to power on / off the display), the caller can define new command such as `VolumeUp`, `VolumeDown`, but the command name can’t contain space. If the specified command has not been set before, it will be added, otherwise the existing command will be modified.
	2. The second parameter indicates the format of the subsequent parameter *CmdStr*. `HEX` means hexadecimal and `STR` means printable string.
	3. The third parameter is the actual command string.
	✧ For hexadecimal format, there is no space between the adjacent bytes. For example:
	`112233445566`
	✧ For string format, the spaces among the words will be regarded as the part of the command string. For example
	`Power on`
	As the factory default, no RS232 message is set.

【Example】

Request:
```
SET UART_CMD ON STR PowerOn
```
Response:
```
UART_CMD ON STR PowerOn
```

2.5.18 Delete RS232 command

【Format】

Request	`SET UART_CMD_D < CmdName >`
Response	`UART_CMD_D < CmdName >`
Description	Delete a defined RS232 command.

【Example】

Request:
```
SET UART_CMD_D ON
```
Response:

```
UART_CMD_D ON
```

2.5.19 Get RS232 command

【Format】

<table>
<tr><td>Request</td><td><code>GET UART_CMD < CmdName | ALL ></code></td></tr>
<tr><td>Response</td><td><code>UART_CMD < CmdName > < HEX | STR > < CmdStr ></code>
Or
<code>UART_CMD ON < HEX | STR > < CmdStr ></code>
<code>UART_CMD OFF < HEX | STR > < CmdStr ></code>
<code>...</code></td></tr>
<tr><td>Description</td><td>Query a RS232 command configuration, if the request message parameter value is <code>ALL</code>, the device returns all defined RS232 messages row by row.</td></tr>
</table>

【Example】

Request:

```
GET UART_CMD ON
```

Response:

```
UART_CMD ON STR PowerOn
```

2.5.20 Send RS232 command

【Format】

<table>
<tr><td>Request</td><td><code>SET UART_CMD_S < CmdName ></code></td></tr>
<tr><td>Response</td><td><code>UART_CMD_S < CmdName ></code></td></tr>
<tr><td>Description</td><td>Send a command through the RS232 port.</td></tr>
</table>

【Example】

Request:

```
SET UART_CMD_S ON
```

Response:

```
UART_CMD_S ON
```

2.5.21 Send RS232 data

【Format】

<table>
<tr><td>Request</td><td><code>SET UART_S < HEX | STR > < CmdStr ></code></td></tr>
<tr><td>Response</td><td><code>UART_S < HEX | STR > < CmdStr ></code></td></tr>
<tr><td>Description</td><td>Send a command through the RS232 port. With this API, the controller can send any RS232 data directly, no need to use the API <code>SET UART_CMD < CmdName > < HEX | STR > < CmdStr ></code> to define a RS232 command in advance.</td></tr>
</table>

Request:

```
SET UART_S STR PowerOn
```

Response:

```
UART_S STR PowerOn
```

2.5.22 Send CEC or/and RS232 command

【Format】

<table>
<tr><td>Request</td><td><code>SET SEND_CMD < ON | OFF | CmdName > [RS232 | CEC | ALL]</code></td></tr>
<tr><td>Response</td><td><code>SEND_CMD < ON | OFF | CmdName > [RS232 | CEC | ALL]</code></td></tr>
<tr><td>Description</td><td>Instruct the device to send CEC or/and RS232 command. The first parameter is the name of sent command, the second optional parameter indicates the sending method. If the sending method is not specified, the configuration of the API <code>SET UARTPWR_FN < ON | OFF ></code> is used.
If the RS232 work mode is not <code>COM</code> or the specified RS232 command is not defined, the device does not send RS232 command.</td></tr>
</table>

【Example】

Request:

```
SET SEND_CMD ON CEC
```

Response:

```
SEND_CMD ON CEC
```

2.5.23 Send CEC data

【Format】

<table>
<tr><td>Request</td><td><code>SET CEC_CMD < OUT1 | OUT2 > < CECCode ></code></td></tr>
<tr><td>Response</td><td><code>CEC_CMD < OUT1 | OUT2 > < CECCode ></code></td></tr>
<tr><td>Description</td><td>Send CEC data directly, this API can send any data through the CEC channel conveniently.</td></tr>
</table>

【Example】

Request:

```
SET CEC_CMD OUT1 4004
```

Response:

```
CEC_CMD OUT1 4004
```

2.6 NDI input & output

2.6.1 Set NDI RX group name

【Format】

<table>
<tr><td>Request</td><td><code>SET NDI_RX_GROUP < GroupName ></code></td></tr>
<tr><td>Response</td><td><code>NDI_RX_GROUP < GroupName ></code></td></tr>
<tr><td>Description</td><td>Configure the group name of the NDI RX module, according to NDI specification, only the NDI units in the same group can discover each other.
As the factory default, the group name is <code>Public</code>.</td></tr>
</table>

【Example】

Request:

```
SET NDI_RX_GROUP MeetingRoom1
```

Response:

```
NDI_RX_GROUP MeetingRoom1
```

2.6.2 Get NDI RX group name

【Format】

Request	`GET NDI_RX_GROUP`
Response	`NDI_RX_GROUP <` *GroupName* `>`
Description	Obtain the group name of the NDI RX module.

【Example】

Request:

```
GET NDI_RX_GROUP
```

Response:

```
NDI_RX_GROUP MeetingRoom1
```

2.6.3 Set NDI RX device name

【Format】

Request	`SET NDI_RX_DEV_NAME <` *DeviceName* `>`
Response	`NDI_RX_DEV_NAME <` *DeviceName* `>`
Description	Configure the device name of the NDI RX module. As the factory default, the device name is `CAMSW100-`*xxxx*`-RX`, *xxxx* is the hexadecimal format of the last two digits of the device MAC address.

【Example】

Request:

```
SET NDI_RX_DEV_NAME CamMixer
```

Response:

```
NDI_RX_DEV_NAME CamMixer
```

2.6.4 Get NDI RX device name

【Format】

Request	`GET NDI_RX_DEV_NAME`
Response	`NDI_RX_DEV_NAME <` *DeviceName* `>`
Description	Obtain the device name of the NDI RX module.

【Example】

Request:

```
GET NDI_RX_DEV_NAME
```

Response:

```
NDI_RX_DEV_NAME CamMixer
```

2.6.5 Get NDI device list

【Format】

<table>
<tr><td>Request</td><td>GET NDI_SCAN_DEV_LIST < TX | RX | ALL ></td></tr>
<tr><td>Response</td><td>NDI_SCAN_DEV_LIST < TX | RX > < DeviceName1 > < IPAddress1 >
NDI_SCAN_DEV_LIST < TX | RX > < DeviceName2 > < IPAddress2 >
...</td></tr>
<tr><td>Description</td><td>Obtain the list of devices scanned in the network. The request message can specify the type of the obtained devices.</td></tr>
</table>

【Example】

Request:

```
GET NDI_SCAN_DEV_LIST TX
```

Response:

```
NDI_SCAN_DEV_LIST TX Team1 169.254.34.71
NDI_SCAN_DEV_LIST TX Team2 169.254.179.80
```

2.6.6 Get NDI channel list

【Format】

<table>
<tr><td>Request</td><td>GET NDI_SCAN_CHN_LIST < Devicen\Name | TX | RX | ALL ></td></tr>
<tr><td>Response</td><td>NDI_SCAN_CHN_LIST < DeviceName > < TX | RX > < ChannelName1 > [ChannelName2 ...]
Or
NDI_SCAN_CHN_LIST < DeviceName1 > < TX | RX > < ChannelName11 > [ChannelName12 ...]
NDI_SCAN_CHN_LIST < DeviceName2 > < TX | RX > < ChannelName21 > [ChannelName22 ...]</td></tr>
<tr><td>Description</td><td>According to NDI specification, one NDI RX or TX module can have multiple channels and each channel corresponds to a video source (TX) or decoder (RX). In fact, most NDI TX or RX modules have only one channel.
This API is used to obtain the channel list of a devices (or a type of devices or all devices). Each row of the response message consists of device name, type and all channels' name.</td></tr>
</table>

【Example】

Request:

```
GET NDI_SCAN_CHN_LIST Team1
```

Response:

```
NDI_SCAN_CHN_LIST Team1 TX Leader Member
```

2.6.7 Set NDI video source

【Format】

<table>
<tr><td>Request</td><td>SET NDI_IN_SRC < 1 | 2 | 3 | 4 > TXDeviceName [ChannelName]</td></tr>
<tr><td>Response</td><td>NDI_IN_SRC < 1 | 2 | 3 | 4 > TXDeviceName [ChannelName]</td></tr>
<tr><td>Description</td><td>Set a NDI (RX) decoder receive and decode the stream of a NDI TX channel. The outputs of NDI decoder is used as the inputs of the subsequent video switcher, namely, NdiSrc (NDIIN1~4).</td></tr>
</table>

	The first parameter of the request message specifies a NDI decoder, the second and third parameter specify a NDI TX channel. If the parameter *ChannelName* is omitted, the first channel will be used.

【Example】

Request:

```
SET NDI_IN_SRC 1 Team1 Leader
```

Response:

```
NDI_IN_SRC 1 Team1 Leader
```

2.6.8 Delete NDI video source

【Format】

Request	`SET NDI_IN_SRC < 1	2	3	4 >`
Response	`NDI_IN_SRC < 1	2	3	4 >`
Description	Delete a NDI video source, the specified NDI (RX) decoder stops receiving or decoding the video stream and the corresponding *NdiSrc* becomes invalid too.

【Example】

Request:

```
SET NDI_IN_SRC 1
```

Response:

```
NDI_IN_SRC 1
```

2.6.9 Get NDI video source

【Format】

Request	`GET NDI_IN_SRC < 1	2	3	4	ALL >`
Response	`NDI_IN_SRC < 1	2	3	4 > [< TXDeviceName > < ChannelName >]` Or `NDI_IN_SRC 1 [< TXDeviceName1 > < ChannelName1 >]` `NDI_IN_SRC 2 [< TXDeviceName2 > < ChannelName2 >]` `NDI_IN_SRC 3 [< TXDeviceName3 > < ChannelName3 >]` `NDI_IN_SRC 4 [< TXDeviceName4 > < ChannelName4 >]`
Description	Get the video source configuration of a certain NDI decoder or all NDI decoders. In the response message, if there is not *TXDeviceName* or *ChannelName* fields, it means the corresponding NDI decoder is not set a NDI source.

【Example】

Request:

```
GET NDI_IN_SRC 1
```

Response:

```
NDI_IN_SRC 1 Team1 Leader
```

2.6.10 Set NDI TX group name

【Format】

Request	SET NDI_TX_GROUP < *GroupName* >
Response	NDI_TX_GROUP < *GroupName* >
Description	Configure the group name of the NDI TX module, according to NDI specification, only the NDI units in the same group can discover each other. As the factory default, the group name is Public.

【Example】

Request:

```
SET NDI_TX_GROUP MeetingRoom1
```

Response:

```
NDI_TX_GROUP MeetingRoom1
```

2.6.11 Get NDI TX group name

【Format】

Request	GET NDI_TX_GROUP
Response	NDI_TX_GROUP < *GroupName* >
Description	Obtain the group name of the NDI TX module.

【Example】

Request:

```
GET NDI_TX_GROUP
```

Response:

```
NDI_TX_GROUP MeetingRoom1
```

2.6.12 Set NDI TX device name

【Format】

Request	SET NDI_TX_DEV_NAME < *DeviceName* >
Response	NDI_TX_DEV_NAME < *DeviceName* >
Description	Configure the device name of the NDI TX module. As the factory default, the device name is CAMSW100-*xxxx*-TX, *xxxx* is the hexadecimal format of the last two digits of the device MAC address.

【Example】

Request:

```
SET NDI_TX_DEV_NAME Teacher
```

Response:

```
NDI_TX_DEV_NAME Teacher
```

2.6.13 Get NDI TX device name

【Format】

Request	GET NDI_TX_DEV_NAME
Response	NDI_TX_DEV_NAME < *DeviceName* >
Description	Obtain the device name of the NDI TX module.

【Example】

Request:

```
GET NDI_TX_DEV_NAME
```

Response:

```
NDI_TX_DEV_NAME Teacher
```

2.6.14 Set NDI TX channel name

【Format】

Request	`SET NDI_TX_CHN_NAME < ChannelName >`
Response	`NDI_TX_CHN_NAME < ChannelName >`
Description	Configure the channel name of the NDI TX module. As the factory default, the device name is `NDI`.

【Example】

Request:

```
SET NDI_TX_CHN_NAME NDIOut1
```

Response:

```
NDI_TX_CHN_NAME NDIOut1
```

2.6.15 Get NDI TX channel name

【Format】

Request	`GET NDI_TX_CHN_NAME`
Response	`NDI_TX_CHN_NAME < ChannelName >`
Description	Obtain the channel name of the NDI TX module.

【Example】

Request:

```
GET NDI_TX_CHN_NAME
```

Response:

```
NDI_TX_CHN_NAME NDIOut1
```

2.6.16 Set NDI TX encoding parameter

【Format】

Request	`SET NDI_TX_VENC_CFG < NDI11 | NDI12 > < EncodeMode > < Resolution > < FrameRate > < BitRateCtl > < BitRate > < GopSize >`
Response	`NDI_TX_VENC_CFG < NDI11 | NDI12 > < EncodeMode > < Resolution > < FrameRate > < BitRateCtl > < BitRate > < GopSize >`
Description	Set the encoding parameter of NDI TX. Here are the details of the parameters: ✧ The first parameter indicates which NDI stream to configure. It takes one of two values: `NDI11` and `NDI12`. Their formats are `NDI` followed by two digits. The first digit is the NDI output number. The device currently supports only one NDI output, so this digit is always `1`. The second digit is used to distinguishes the main stream from

<table>
<tr><td></td><td>the preview stream, 1 and 2 correspond to the main stream and preview stream respectively.
✧ EncodeMode Encoding method, allowed values are H.264 and H.265.
✧ Resolution Encoding resolution, the optional values varies on the stream:
Main stream(NDI11): 1280×720, 1920×1080, 3840×2160
Preview stream(NDI12): 320×240, 640×360
✧ FrameRate Encoding frame rate, the range is 1 to 60 and the unit is fps.
✧ BitRateCtl Bitrate-control mode, CBR or VBR.
✧ BitRate Bitrate, the unit is kbps, the range varies on the stream:
Main stream(NDI11): 64~16384
Preview stream(NDI12): 64~4096
✧ GopSize Group-of-Pictures size, i.e., I-frame interval, the range is 1~150.
Here are the factory defaults:
Main stream(NDI11):
<code>H.264 1920x1080 30 cbr 4096 30</code>
Preview stream(NDI12):
<code>H.264 320x240 30 cbr 1024 30</code>
Note:
Currently, The frame rate cannot be set arbitrarily, it must be determined automatically by the encoding module, so the parameter FrameRate is ignored now.</td></tr>
</table>

【Example】

Request:

```
SET NDI_TX_VENC_CFG NDI11 H.264 1920X1080 30 CBR 8192 60
```

Response:

```
NDI_TX_VENC_CFG NDI11 H.264 1920X1080 30 CBR 8192 60
```

2.6.17 Get NDI TX encoding parameter

【Format】

<table>
<tr><td>Request</td><td><code>SET NDI_TX_VENC_CFG < NDI11 | NDI12 | ALL ></code></td></tr>
<tr><td>Response</td><td>NDI_TX_VENC_CFG < NDI11 | NDI12 > < EncodeMode > < Resolution > < FrameRate > < BitRateCtl > < BitRate > < GopSize >
Or
NDI_TX_VENC_CFG NDI11 < EncodeMode > < Resolution > < FrameRate > < BitRateCtl > < BitRate > < GopSize >
NDI_TX_VENC_CFG NDI12 < EncodeMode > < Resolution > < FrameRate > < BitRateCtl > < BitRate > < GopSize ></td></tr>
<tr><td>Description</td><td>Get the encoding parameter of NDI TX, if the parameter value of the request message is ALL, the device returns with two rows corresponding to the two streams.</td></tr>
</table>

【Example】

Request:

```
GET NDI_TX_VENC_CFG NDI11
```

Response:

```
NDI_TX_VENC_CFG NDI11 H.264 1920X1080 30 CBR 8192 60
```

2.6.18 Set NDI stream output

【Format】

<table>
<tr><td>Request</td><td><code>SET NDI_TX_STREAM_FN < NDI11 | NDI12 > < ON | OFF ></code></td></tr>
<tr><td>Response</td><td><code>NDI_TX_STREAM_FN < NDI11 | NDI12 > < ON | OFF ></code></td></tr>
<tr><td>Description</td><td>Set the NDI TX stream output feature, namely, turn on/off the NDI stream output. <code>NDI11</code> and <code>NDI12</code> correspond to main stream and preview stream respectively.
As the factory default, both two streams are enabled, the configuration value is <code>ON</code>.</td></tr>
</table>

【Example】

Request:

```
SET NDI_TX_STREAM_FN NDI12 OFF
```

Response:

```
NDI_TX_STREAM_FN NDI12 OFF
```

2.6.19 Get NDI stream output

【Format】

<table>
<tr><td>Request</td><td><code>GET NDI_TX_STREAM_FN < NDI11 | NDI12 | ALL ></code></td></tr>
<tr><td>Response</td><td><code>NDI_TX_STREAM_FN < NDI11 | NDI12 > < ON | OFF ></code>
Or
<code>NDI_TX_STREAM_FN NDI11 < ON | OFF ></code>
<code>NDI_TX_STREAM_FN NDI12 < ON | OFF ></code></td></tr>
<tr><td>Description</td><td>Obtain the status of the NDI TX stream output feature.</td></tr>
</table>

【Example】

Request:

```
GET NDI_TX_STREAM_FN NDI12
```

Response:

```
NDI_TX_STREAM_FN NDI12 OFF
```

3 Appendix

[To be added]